claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\workflows\wf_8d68d336-288\agent-a443ef7f8d7dfe16b.jsonl`
- SHA-256: `9c24e2bc9534c58257f2f4dd5da3fe4a6d83cd9cf8abadf0957c98d7bad8bd36`
- Source modified: `2026-07-03T19:52:14+00:00`
- Imported at: `2026-07-08T16:00:15+00:00`
- project: `wf_8d68d336-288`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-03T19:48:21.204Z)

```
Repo (my in-progress optimization branch): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/fix-worktree
Key files to Read:
- backend/pipeline/detectors/wasb_infer.py  ? the inference loop (DataLoader ? detector.run_tensor ? tracker). Contains the streaming SequentialFrameStore, the _PrefetchIter I just added, build_windows, and the tracker re-run loop.
- backend/pipeline/detectors/native_wasb_runner.py ? builds the wasb_infer.py argv on the GPU box (native runner). Has device/stream_video/batch_size/prefetch_batches config.
- backend/config/models.py (WasbIndexConfig) and backend/config/presets.py (cloud-serving preset).
- deploy/cloudrun/Dockerfile ? the WASB conda env is torch 1.11.0+cu113 (py3.8), a SEPARATE env from the app's torch. The detector shells out to it as a subprocess.
The WASB model itself lives at /opt/models/WASB-SBDT (nttcom/WASB-SBDT) ? you can't read it, but reason from how wasb_infer.py calls build_detector(cfg) / detector.run_tensor(imgs, trans) and build_tracker(cfg).

MEASURED BASELINE (first real L4 cloud run, 2026-07-03): a ~7.5-min 60fps badminton clip took Phase-2 (WASB detection) 727.9s (~0.6x realtime); GPU utilization sat <1% the whole time. Hardware: NVIDIA L4 (Ada sm_89, 23GB), 8 vCPU, 32GB. Streaming mode forces DataLoader num_workers=0 (single-process), so CPU batch assembly (cv2 decode + PIL + ToTensor) serializes with GPU inference.

CRITICAL CONSTRAINT ? the D5/D16 byte-parity guarantee: the repo REQUIRES that the rally/window OUTPUT (the candidate windows and trajectory CSV) stay bit-identical / deterministic across runs and across the disk-frames vs streaming paths. This is asserted by a real parity test. An optimization that changes detector NUMERICS (FP16, TF32, cudnn.benchmark algo selection, NVDEC vs CPU decode) risks changing the output and breaking that guarantee. Batch size and prefetch do NOT change per-frame numerics (same forward, just grouped/overlapped) so they are parity-safe.
Already implemented on this branch: indexing.wasb.batch_size + prefetch_batches knobs, the cloud-serving preset opts into batch 64 + prefetch 4.


You are a CUDA/PyTorch performance engineer. Investigate ONE optimization lever in depth by READING the actual code. Lever "model-warmup-persist":
Investigate two container-lifecycle levers: (1) a warmup forward pass at healthcheck time so the FIRST real batch does not pay lazy CUDA context + cudnn autotune + JIT cost mid-measurement; (2) whether the ephemeral Cloud Run Job model means EVERY job reloads weights + re-JITs ? would a persistent Cloud Run SERVICE (or a torch.jit.trace/save of the model cached in the image) amortize that. Parity-safe framing. Also: is torch.set_grad_enabled already off (torch.no_grad is used) and is model.eval() set?

Read the named files, reason from the real code, and return the structured finding. Be concrete
```

about code_locations (file:function). Be honest about parity_risk ? the byte-parity guarantee is load-bearing.

2. assistant (2026-07-03T19:48:23.230Z)

I'll investigate the container-lifecycle warmup/persist levers by reading the actual code.

3. user (2026-07-03T19:48:26.088Z)

```
"""WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).

WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
own building blocks (ImageDataset transform + detector + online tracker) to run on
any folder of extracted frames and emit a trajectory CSV in the exact shape the
indexer's `tracknet_runner.parse_trajectory_csv` already consumes:

    Frame,Visibility,X,Y

It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
(`wasb_runner.py`) stages this file + the frames into WSL and invokes it.

Resumable across reboots
------------------------
The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
the slow, expensive part; the CPU tracker pass that turns detections into a
trajectory is cheap and *stateful* (it must see every frame in order, so it can't
resume mid-stream). We exploit that split:

  * The detector pass writes its per-window raw outputs incrementally to
    ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
    records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
    most ``--flush-every`` windows of GPU work instead of the whole pass.
  * On restart we replay the cached windows, skip the DataLoader ahead to the first
    un-cached window, and continue. Once every window is cached, the detector pass
    is skipped entirely and we just re-run the (cheap, deterministic) tracker over
    the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
    not the GPU hour.

The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
weights, sport, window size, or frame count change.

Usage (in WSL, from ~/models/WASB-SBDT/src):
    python wasb_infer.py --frames_dir /path/frames --weights ../pretrained_weights/wasb_badminton_best.pth.tar \
        --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-every 1000] [--fresh]
"""

import argparse
import csv
import glob
import json
import logging
import os
import os.path as osp
from collections import defaultdict
from contextlib import contextmanager

logger = logging.getLogger(__name__)

CACHE_VERSION = 1
_DET_FILE = "det_raw.jsonl"
_MANIFEST_FILE = "manifest.json"
```

```


def _build_cfg(weights: str, sport: str, device: str = "cuda"):
    """Compose the WASB eval config via Hydra, overriding model/weights/device.

    ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
    ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests ``runner.gpus=[0]``;
    cpu/mps run with no GPU list so the detector places tensors on ``device``.
    """
    from hydra import compose, initialize

    gpus = "[0]" if device == "cuda" else "[]"  # single GPU on cuda; none for cpu/mps
    with initialize(config_path="configs", version_base=None):
        cfg = compose(
            config_name="eval",
            overrides=[
                f"dataset={sport}",
                "model=wasb",
                f"detector.model_path={weights}",
                f"runner.device={device}",
                f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to this box
            ],
        )
    return cfg


def _frame_id(path: str) -> int:
    """Best-effort integer frame id from a frame filename (e.g. frame_000180.png -> 180)."""
    stem = osp.splitext(osp.basename(path))[0]
    digits = "".join(ch for ch in stem if ch.isdigit())
    return int(digits) if digits else -1


# ------------------------------------------------------------------------ #
# Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
# ------------------------------------------------------------------------ #
def _cache_paths(cache_dir: str):
    return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


def default_cache_dir(out_csv: str) -> str:
    """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
    d = osp.dirname(osp.abspath(out_csv))
    stem = osp.splitext(osp.basename(out_csv))[0]
    return osp.join(d, stem + "_wasbcache")


def _atomic_write_text(path: str, text: str) -> None:
    """Write then rename so a crash mid-write can't leave a half-written file."""
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        f.write(text)
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, path)


def _det_plain(det) -> list:
    """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able list."""
    xy = det["xy"]
```

```
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


def _dets_to_tracker(plain_list):
    """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy array,
    the tracker does vector math / np.linalg.norm on it)."""
    import numpy as np

    return [
        {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
        for p in plain_list
    ]


def write_manifest(cache_dir: str, manifest: dict) -> None:
    _, mpath = _cache_paths(cache_dir)
    _atomic_write_text(mpath, json.dumps(manifest, indent=2))


def read_manifest(cache_dir: str):
    _, mpath = _cache_paths(cache_dir)
    if not osp.exists(mpath):
        return None
    try:
        with open(mpath) as f:
            return json.load(f)
    except (json.JSONDecodeError, OSError):
        return None


def manifest_compatible(
    manifest: dict,
    *,
    frames_dir: str,
    weights: str,
    sport: str,
    frames_in: int,
    frames_total: int,
    device: str = "cuda",
) -> bool:
    """A cache is reusable only if the run that produced it matches this run's inputs.

    ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
    (detections can differ numerically across devices), and vice-versa.

    ``frames_dir`` is the **already-canonical cache key** the caller stored in the manifest
    (``run()`` passes its ``cache_key`` here, and writes the same value into the manifest):
    ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic ``video:<abspath>``
    key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath`` it here:
    ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD prepended,
    so a streaming cache would never match itself and every re-run would recompute (issue #348).
    """
    if not manifest or manifest.get("version") != CACHE_VERSION:
        return False
    return (
```

```
        manifest.get("frames_dir") == frames_dir
        and manifest.get("weights") == weights
        and manifest.get("sport") == sport
        and manifest.get("frames_in") == frames_in
        and manifest.get("frames_total") == frames_total
        and manifest.get("device", "cuda") == device
    )


def reset_cache(cache_dir: str) -> None:
    """Drop any existing cache so the next run starts clean."""
    det_jsonl, mpath = _cache_paths(cache_dir)
    for p in (det_jsonl, mpath):
        if osp.exists(p):
            os.remove(p)


def load_and_clean_cache(cache_dir: str):
    """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
    rebuild the per-frame detection accumulation, and rewrite the file to exactly that
    prefix so a trailing half-written line from a crash can't corrupt future appends.

    Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
    [x, y, score, scale] candidates (accumulated across every window the frame is in,
    in window order ? identical to a non-resumed run).
    """
    det_jsonl, _ = _cache_paths(cache_dir)
    det_by_fid = defaultdict(list)
    valid: list = []
    if osp.exists(det_jsonl):
        with open(det_jsonl, "r") as f:
            for line in f:
                s = line.strip()
                if not s:
                    continue
                try:
                    obj = json.loads(s)
                except json.JSONDecodeError:
                    break  # truncated trailing line (crash mid-flush)
                if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                    break
                valid.append(s)
                for fid, plain in obj["f"]:
                    det_by_fid[int(fid)].extend([list(p) for p in plain])
        # Guarantee a clean append boundary by rewriting only the good prefix.
        _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
    return len(valid), det_by_fid


def _append_windows(fh, objs) -> None:
    """Append window records as JSONL and durably flush (bounded loss on crash)."""
    for o in objs:
        fh.write(json.dumps(o, separators=(",", ":")) + "\n")
    fh.flush()
    os.fsync(fh.fileno())


def _atomic_write_trajectory(out_csv: str, rows) -> None:
    os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
    tmp = out_csv + ".tmp"
    with open(tmp, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["Frame", "Visibility", "X", "Y"])
        w.writerows(rows)
        f.flush()
```

```
        os.fsync(f.fileno())
    os.replace(tmp, out_csv)


# --------------------------------------------------------------------------- #
# Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
# --------------------------------------------------------------------------- #
_STREAM_FRAME_FMT = "{:08d}.png"


def synthetic_frame_path(index: int) -> str:
    """Stable, index-encoding frame name used by the streaming path.

    It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
    downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
    frames came from disk or from the in-memory stream. The streaming image loader
    inverts this name back to an index to fetch the decoded frame.
    """
    return _STREAM_FRAME_FMT.format(int(index))


def build_windows(frames: list, frames_in: int):
    """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
    + checkpoint). Pure list math, identical for the disk and streaming paths.

    Returns a list of windows, each a list of ``frames_in`` consecutive paths
    (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
    caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
    ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
    """
    return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]


class SequentialFrameStore:
    """In-memory sliding buffer over a forward-only frame reader, sized for the
    detector's sliding-window access pattern (peak O(window) frames, not O(video)).

    The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
    ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
    in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
    start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
    boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
    far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
    earliest index any not-yet-finished window can still ask for) and decode each source
    frame exactly once via the forward-only ``reader(index)``.
    """

    def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
        self._reader = reader
        self._total = int(total)
        self._window = max(1, int(window))
        self._buf: dict = {}
        # On a resumed run the detector's first needed frame is `start_index`; begin
        # pulling there so the reader can skip (seek past) the already-cached prefix
        # instead of decoding 0..start_index-1 just to throw them away.
        self._next = int(start_index)  # next index not yet pulled from the reader
        self._max_seen = int(start_index) - 1
        self._floor = int(start_index)  # lowest index we still keep (never evict below)

    def get(self, index: int):
        index = int(index)
        if index < self._floor:
            raise KeyError(
                f"frame {index} already evicted (below floor {self._floor}); the access
"
```

```
                f"pattern must stay within a window of the highest frame seen"
            )
        if index >= self._total:
            raise KeyError(f"frame {index} out of range (total {self._total})")
        # Pull forward from the reader until the requested index is buffered.
        while self._next <= index:
            self._buf[self._next] = self._reader(self._next)
            self._next += 1
        if index > self._max_seen:
            self._max_seen = index
        # Evict frames older than the earliest a still-running window could re-request:
        # once we've seen index `m`, no future window starts before `m - (window-1)`.
        new_floor = max(self._floor, self._max_seen - (self._window - 1))
        for k in range(self._floor, new_floor):
            self._buf.pop(k, None)
        self._floor = new_floor
        return self._buf[index]


def _index_from_synthetic_path(path: str) -> int:
    """Invert ``synthetic_frame_path``: a frame path -> its integer source index.

    Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
    in lockstep with the on-disk naming scheme.
    """
    return _frame_id(path)


def _bgr_to_pil_rgb(frame_bgr):
    """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the EXACT
    PIL RGB image WASB's disk path produces.

    The disk path is ``frame_bgr -> cv2.imwrite(PNG) -> Image.open(PNG).convert('RGB')``;
    because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr, COLOR_BGR2RGB)``
    bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the streamed
    frame indistinguishable from the on-disk one to everything downstream.
    """
    import cv2
    from PIL import Image

    return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))


def _make_streamed_read_image(store, real_read_image):
    """Build the ``read_image`` replacement used during a streaming detector pass.

    A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB image,
    matching ``read_image``'s real return type); any path that doesn't parse to an index falls
    through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-testable
    without the WSL-only ``dataloaders`` package.
    """

    def _read_image(path, *args, **kwargs):
        idx = _index_from_synthetic_path(path)
        if idx < 0:
            return real_read_image(path, *args, **kwargs)
        return _bgr_to_pil_rgb(store.get(idx))

```

```
    return _read_image


@contextmanager
def _streaming_read_image_patch(
    frame_loader, total: int, window: int = 1, start_index: int = 0
):
    """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory decoded
    frames during the detector pass, byte-for-byte as it would over real PNGs.

    WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
    ``cv2.imread``. ``read_image`` (``utils.utils``) does ``Image.open(path).convert('RGB')``,
    returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore feed it a
    PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
    byte-identical to the disk round-trip, and the shim never touches the filesystem so the
    ``osp.exists`` guard is sidestepped for synthetic stream paths.

    We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer actually
    calls (``dataset_loader`` does ``from utils import read_image``, so the name lives in that
    module's namespace; patching ``utils.read_image`` would NOT rebind the already-imported
    reference).

    ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder wrapped
    in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``); on a
    resume we start at ``start_index`` so the decoder seeks past already-cached windows. The
    original reader is restored on exit.
    """
    from dataloaders import dataset_loader as _dl

    store = SequentialFrameStore(
        frame_loader, total, window=window, start_index=start_index
    )
    real_read_image = _dl.read_image
    # Rebind read_image in the consuming module for the duration of the detector pass;
    # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff B010-clean.)
    _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type: ignore[assignment]
    try:
        yield store
    finally:
        _dl.read_image = real_read_image  # type: ignore[assignment]


# ------------------------------------------------------------------------ #
# Inference
# ------------------------------------------------------------------------ #
class _PrefetchIter:
    """Bounded producer-thread wrapper over a batch iterator (the num_workers=0 DataLoader).

    In streaming mode the DataLoader must stay single-process (the in-memory frame store +
    the cv2.imread shim live in this process and cannot be pickled to workers), so the
    CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
```

```
    measured on the first real L4 serving run (2026-07-03) this left the GPU <1% utilized.
    A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
    release the GIL for the heavy parts, the store's forward-only access pattern is
    preserved (one consumer thread iterating sequentially), and the bounded queue caps
    memory at ``depth`` batches. A producer exception is re-raised in the consumer."""

    _END = object()

    def __init__(self, src, depth: int = 2):
        import queue
        import threading

        self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
        self._exc: "BaseException | None" = None

        def _fill() -> None:
            try:
                for item in src:
                    self._q.put(item)
            except BaseException as e:  # noqa: BLE001 - surfaced in __next__
                self._exc = e
            finally:
                self._q.put(self._END)

        self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
        self._t.start()

    def __iter__(self):
        return self

    def __next__(self):
        item = self._q.get()
        if item is self._END:
            if self._exc is not None:
                raise self._exc
            raise StopIteration
        return item


def run(
    frames,
    weights: str,
    out_csv: str,
    sport: str = "badminton",
    limit: int = 0,
    batch_size: int = 8,
    num_workers: int = 4,
    log_every_batches: int = 50,
    cache_dir: "str | None" = None,
    flush_every: int = 1000,
    fresh: bool = False,
    frame_loader=None,
    source_key: "str | None" = None,
    device: str = "cuda",
    prefetch_batches: int = 0,
) -> str:
    """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.

    Two equivalent ways to supply pixels (output is bit-identical between them):

    * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
      globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via ``cv2.imread``.
    * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
```

```
      frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
      pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
      reader, which returns a PIL RGB image) over the DataLoader pass so every other line of
      ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is identical
      to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
      ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces ``num_workers=0``
      (the in-process shim + buffer can't cross worker processes).

    ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
    frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
    after reboot still matches.
    """
    import time

    import torch
    from dataloaders import build_img_transforms, build_seq_transforms
    from dataloaders.dataset_loader import ImageDataset
    from torch.utils.data import DataLoader

    # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside the
    # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore` suppresses
    # the mypy false positive where the main env resolves `trackers` to the Roboflow package
    # (which has no `build_tracker`). There is NO runtime collision ? this runs in a separate
    # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow dep.
    from trackers import build_tracker  # type: ignore[attr-defined]
    from utils import Center

    from detectors import build_detector

    streaming = frame_loader is not None

    cfg = _build_cfg(weights, sport, device)
    frames_in = int(cfg["model"]["frames_in"])
    input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
    output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))

    if streaming:
        # `frames` is already the ordered list of (synthetic) frame paths.
        if not isinstance(frames, (list, tuple)):
            raise SystemExit(
                "streaming mode requires `frames` to be a list of frame paths"
            )
        frames = list(frames)
        frames_dir = source_key or "stream"
    else:
        frames_dir = frames
        frames = sorted(
            glob.glob(osp.join(frames_dir, "*.png"))
            + glob.glob(osp.join(frames_dir, "*.jpg"))
        )
    if limit:
        frames = frames[:limit]
    if len(frames) < frames_in:
        where = frames_dir if not streaming else "video stream"
        raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")

    # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
```

```
checkpoint).
        samples = []
        for window in build_windows(frames, frames_in):
            annos = [
                {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
                for p in window
            ]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {},
            frames_dir=cache_key,
            weights=weights,
            sport=sport,
            frames_in=frames_in,
            frames_total=len(frames),
            device=device,
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(
                f"resuming from cache: {windows_done}/{windows_total} windows already detected"
            )
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": cache_key,
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "device": device,
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ----------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
                seq_transform_test = None
            ds = ImageDataset(
                cfg,
                remaining,
                input_wh,
                output_wh,
                transform=transform_test,
```

```
            seq_transform=seq_transform_test,
            is_train=False,
        )
        # Streaming forces single-process loading: the in-memory frame store + the
        # cv2.imread shim live in THIS process and can't be pickled to DataLoader workers.
        loader_workers = 0 if streaming else num_workers
        loader = DataLoader(
            ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
        )
        detector = build_detector(cfg)

        from contextlib import ExitStack

        t0 = time.time()
        done = windows_done
        win_idx = windows_done
        log_every = max(1, log_every_batches)
        flush_n = max(1, flush_every)
        pending = []
        logger.info(
            f"detecting on {len(remaining)} remaining windows "
            f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
            f"source={'video-stream' if streaming else 'frames-dir'})..."
        )
        det_f = open(det_jsonl, "a")
        try:
            with ExitStack() as stack:
                stack.enter_context(torch.no_grad())
                # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)` to the
                # in-memory decoded frame for that synthetic path; every other line of
                # __getitem__ runs unchanged so the tensor matches the PNG round-trip exactly.
                # The detector's first needed frame is `windows_done` (window i starts at
                # frame i), so prime the store there for a resume.
                if streaming:
                    stack.enter_context(
                        _streaming_read_image_patch(
                            frame_loader,
                            len(frames),
                            window=frames_in,
                            start_index=windows_done,
                        )
                    )
                # Overlap CPU batch assembly with GPU inference (streaming/workers=0 only ?
                # a multi-worker DataLoader already overlaps via its worker processes).
                batches = (
                    _PrefetchIter(loader, depth=prefetch_batches)
                    if prefetch_batches and loader_workers == 0
                    else loader
                )
                for bi, batch in enumerate(batches):
                    imgs, _hms, trans = batch[0], batch[1], batch[2]
                    img_paths = [
                        list(t) for t in batch[-1]
                    ]  # [frame_in][batch] -> path
                    batch_results, _ = detector.run_tensor(imgs, trans)
                    nb = imgs.shape[0]
                    for ib in range(nb):
                        frames_payload = []
                        for ie in sorted(batch_results[ib].keys()):
                            p = img_paths[ie][ib]
```

```
                            fid = _frame_id(p)
                            plain = [_det_plain(d) for d in batch_results[ib][ie]]
                            frames_payload.append([fid, plain])
                            det_by_fid[fid].extend(plain)
                        pending.append({"w": win_idx, "f": frames_payload})
                        win_idx += 1
                    done += nb
                    if len(pending) >= flush_n or done >= windows_total:
                        _append_windows(det_f, pending)
                        pending = []
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
                    if (bi + 1) % log_every == 0 or done >= windows_total:
                        el = time.time() - t0
                        new_done = done - windows_done
                        rate = new_done / el if el > 0 else 0.0
                        eta = (windows_total - done) / rate if rate > 0 else 0.0
                        logger.info(
                            f"{done}/{windows_total} "
                            f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
                            f"elapsed {el:.0f}s | ETA {eta:.0f}s"
                        )
                if pending:
                    _append_windows(det_f, pending)
                    base_manifest["windows_done"] = done
                    write_manifest(cache_dir, base_manifest)
        finally:
            det_f.close()
        logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
    else:
        logger.info("detector cache already complete -> tracker-only re-run")

    # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
    tracker = build_tracker(cfg)
    tracker.refresh()
    rows = []
    for fid in sorted(det_by_fid.keys()):
        r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
        rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

    _atomic_write_trajectory(out_csv, rows)
    visible = sum(1 for _, v, *_ in rows if v)
    logger.info(f"wrote {len(rows)} rows -> {out_csv}")
    logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
    return out_csv


def extract_frames(video_path: str, frames_dir: str) -> str:
    """Decode a video to PNG frames named by source frame index (00000000.png ...).

    Idempotent: if the dir already holds the expected number of frames (matching the
    container's reported frame count) we skip re-decoding ? re-extraction after a
    reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
    extracting with ffmpeg on the host and passing --frames_dir.
    """
    import cv2

    os.makedirs(frames_dir, exist_ok=True)
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise SystemExit(f"cannot open video: {video_path}")
    expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    existing = len(glob.glob(osp.join(frames_dir, "*.png")))
    if expected > 0 and existing >= expected:
```

```
        cap.release()
        logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
        return frames_dir
    i = 0
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
        i += 1
    cap.release()
    logger.info(f"extracted {i} frames -> {frames_dir}")
    return frames_dir


def _probe_frame_count(video_path: str) -> int:
    """Container-reported frame count via cv2 (0 if unknown). Used to size the stream."""
    import cv2

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise SystemExit(f"cannot open video: {video_path}")
    n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    cap.release()
    return n


def make_video_frame_loader(video_path: str):
    """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

    ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
    long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
    detector's sliding-window order); it reads sequentially and only uses a frame-seek to
    skip forward when a *resume* starts past the current position. Each ``cap.read()``
    returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG would
    yield (PNG is lossless), so the detector sees identical pixels to the disk path.

    Returns the count hint from the container so the caller can build the frame list;
    ``release`` closes the capture.
    """
    import cv2

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise SystemExit(f"cannot open video: {video_path}")
    count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    state = {"pos": 0}  # next index cap.read() will return

    def frame_loader(index: int):
        index = int(index)
        if index < state["pos"]:
            raise RuntimeError(
                f"streaming decode is forward-only; got index {index} after {state['pos']}"
            )
        if index > state["pos"]:
            # Forward skip only happens when resuming past cached windows. Seek once.
            cap.set(cv2.CAP_PROP_POS_FRAMES, index)
            state["pos"] = index
        ok, frame = cap.read()
        if not ok:
            raise RuntimeError(f"failed to decode frame {index} from {video_path}")
```

```
            state["pos"] += 1
            return frame

        def release():
            cap.release()

        return frame_loader, count_hint, release


    def run_video_streaming(
        video_path: str,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
        fresh: bool = False,
        device: str = "cuda",
        prefetch_batches: int = 0,
    ) -> str:
        """Fast path: decode ``video_path`` on the fly and feed frames straight to the
        detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
        dominates a long clip). Output is bit-identical to the frames-on-disk path; see
        ``run``'s docstring for why.
        """
        frame_loader, count_hint, release = make_video_frame_loader(video_path)
        try:
            if count_hint <= 0:
                raise SystemExit(
                    f"could not determine frame count for {video_path}; cannot stream "
                    f"(fall back to frame extraction with --frames_dir)"
                )
            frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
            source_key = "video:" + osp.abspath(video_path)
            return run(
                frame_list,
                weights,
                out_csv,
                sport,
                limit,
                batch_size=batch_size,
                num_workers=0,
                log_every_batches=log_every_batches,
                cache_dir=cache_dir,
                flush_every=flush_every,
                fresh=fresh,
                frame_loader=frame_loader,
                source_key=source_key,
                device=device,
                prefetch_batches=prefetch_batches,
            )
        finally:
            release()


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument(
            "--video", help="video file; frames are extracted internally (in WSL)"
        )
        ap.add_argument(
```

```
        "--frames_out_dir", help="where to extract frames when --video is used"
    )
    ap.add_argument(
        "--stream-video",
        action="store_true",
        help="FAST PATH: decode --video on the fly and feed frames straight to "
        "the detector (no PNG extraction). Output-equivalent to the disk path.",
    )
    ap.add_argument("--weights", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--sport", default="badminton")
    ap.add_argument(
        "--device",
        default="cuda",
        choices=["cuda", "mps", "cpu"],
        help="torch device for the detector (default cuda = the WSL parity path)",
    )
    ap.add_argument(
        "--limit", type=int, default=0, help="cap #frames (for quick tests)"
    )
    ap.add_argument("--batch-size", type=int, default=8)
    ap.add_argument("--num-workers", type=int, default=4)
    ap.add_argument(
        "--prefetch-batches",
        type=int,
        default=0,
        help="streaming mode: overlap CPU batch assembly with GPU inference via a "
        "bounded producer thread of this queue depth (0 = off, the parity default)",
    )
    ap.add_argument(
        "--log-every-batches", type=int, default=50, help="progress log cadence"
    )
    ap.add_argument(
        "--cache-dir",
        default=None,
        help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
    )
    ap.add_argument(
        "--flush-every",
        type=int,
        default=1000,
        help="fsync the detector cache every N windows (bounds reboot loss)",
    )
    ap.add_argument(
        "--fresh", action="store_true", help="ignore any existing cache and start over"
    )
    args = ap.parse_args()

    if args.stream_video:
        if not args.video:
            raise SystemExit("--stream-video requires --video")
        run_video_streaming(
            args.video,
            args.weights,
            args.out,
            args.sport,
            args.limit,
            batch_size=args.batch_size,
            log_every_batches=args.log_every_batches,
            cache_dir=args.cache_dir,
            flush_every=args.flush_every,
            fresh=args.fresh,
            device=args.device,
            prefetch_batches=args.prefetch_batches,
        )
```

```
        return

    frames_dir = args.frames_dir
    if args.video:
        frames_dir = extract_frames(
            args.video, args.frames_out_dir or (args.video + "_frames")
        )
    if not frames_dir:
        raise SystemExit("provide --frames_dir or --video")
    run(
        frames_dir,
        args.weights,
        args.out,
        args.sport,
        args.limit,
        batch_size=args.batch_size,
        num_workers=args.num_workers,
        log_every_batches=args.log_every_batches,
        cache_dir=args.cache_dir,
        flush_every=args.flush_every,
        fresh=args.fresh,
        device=args.device,
    )


if __name__ == "__main__":
    main()

```

4. user (2026-07-03T19:48:26.731Z)

```
"""Linux-native WASB shuttle-trajectory runner (no WSL).

The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
"Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md`` M3.5).
It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py`` core
that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float tolerance;
see the acceptance gate below) ? but WITHOUT the WSL transport:

  * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
    ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
  * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or ``auto``
    (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit ``cuda`` stays
    strict); threaded into ``wasb_infer.py --device`` (and so the Hydra ``runner.device`` override).
  * weights / repo / python / scratch come from the **environment** first
    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
    ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.

It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
(``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``); ``detector_impl="native"``.

**Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL runner
```

```
(Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the same
candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-deterministic
ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-exact
run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native streaming
vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
artifact), and two native runs were byte-identical (deterministic). This is a hardware check
(a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the contract,
not the numbers.
"""

import logging
import os
import shutil
import subprocess
from dataclasses import dataclass
from typing import List, Optional, Tuple

from backend.config.models import IndexingConfig
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext

# Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to WSL/TrackNet.
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

logger = logging.getLogger(__name__)

# Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
# detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
_INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")

# Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution probe
# so the "cuda True/False" string they parse can never drift apart.
_CUDA_PROBE_CODE = (
    "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
)


@dataclass
class NativeWasbConfig:
    """Resolved settings for a Linux-native WASB run.

    Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
    over ``indexing.wasb.*`` config (the box sets env; no config edits needed). ``device``
    defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the perf win).
    """

    repo_dir: str = "~/models/WASB-SBDT"  # native clone of nttcom/WASB-SBDT
    python_bin: str = (
        "python"  # the `wasb` conda env python (invoked by path, no activate)
    )
    weights_path: str = ""  # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
    sport: str = "badminton"
    device: str = "cuda"  # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
    scratch_dir: str = ""  # frame-extraction scratch (env); "" = next to the output dir
    timeout_sec: int = 1800  # 30 min; a hung GPU call is killed (0 = no timeout)
```

```
    keep_frames: bool = (
        False  # retain the decoded frame cache after success (debug only)
    )
    # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long clip's
    # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a real GPU
    # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default here.
    stream_video: bool = True
    # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
    # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset opts in.
    batch_size: int = 0
    prefetch_batches: int = 0

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
        if isinstance(idx_cfg, dict):
            idx_cfg = IndexingConfig(**idx_cfg)
        w = idx_cfg.wasb
        env = os.environ.get
        # stream_video is tri-state in config: None/absent => native default (stream); an
        # explicit True/False still wins (e.g. False for a container cv2 can't seek).
        sv = w.stream_video
        return cls(
            repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
            python_bin=env("WASB_PYTHON") or w.python_bin,
            weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
            sport=w.sport,
            device=env("WASB_DEVICE") or w.device,
            scratch_dir=env("WASB_SCRATCH")
            or env("RALLY_SCRATCH")
            or env("TMPDIR")
            or "",
            timeout_sec=w.timeout_sec,
            keep_frames=w.keep_frames,
            stream_video=(True if sv is None else bool(sv)),
            batch_size=int(w.batch_size or 0),
            prefetch_batches=int(w.prefetch_batches or 0),
        )


class NativeWasbRunner(DetectorRunner):
    """WASB runner that executes natively on a Linux GPU box ? no WSL. See module docstring."""

    def __init__(self, cfg: NativeWasbConfig):
        self.cfg = cfg
        # Cached CUDA-availability probe (used to resolve device="auto"). Set by healthcheck;
        # lazily probed by run_predict if healthcheck was skipped. None = not yet probed.
        self._cuda_available: Optional[bool] = None

    # --- low-level native exec (the single place tests patch) --------------------------
    def _run(
        self, argv: List[str], cwd: Optional[str] = None
    ) -> subprocess.CompletedProcess:
        logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
        return subprocess.run(
            argv,
            cwd=cwd,
```

```
            capture_output=True,
            text=True,
            timeout=(self.cfg.timeout_sec or None),
        )

    # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-cpu) ------
    def _probe_cuda(self) -> bool:
        """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess). Any failure
        (missing python / timeout / import error) is treated as 'no CUDA'."""
        try:
            res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
        except (FileNotFoundError, subprocess.TimeoutExpired):
            return False
        return res.returncode == 0 and "cuda True" in (res.stdout or "")

    def _cuda_is_available(self) -> bool:
        """CUDA availability, cached across healthcheck + run_predict so we probe at most once."""
        if self._cuda_available is None:
            self._cuda_available = self._probe_cuda()
        return self._cuda_available

    def _effective_device(self) -> str:
        """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs the CUDA
        probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is unchanged)."""
        if self.cfg.device == AUTO:
            return DeviceContext(AUTO, self._cuda_is_available()).effective
        return self.cfg.device

    def _repo_src(self) -> str:
        return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")

    def _copy_infer_script(self) -> bool:
        """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds configs/."""
        repo_src = self._repo_src()
        try:
            os.makedirs(repo_src, exist_ok=True)
            shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
            return True
        except OSError as e:
            logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
            return False

    def _rm_rf(self, path: Optional[str]) -> None:
        """Best-effort recursive delete of a native path (post-success cleanup; never raises)."""
        if path:
            shutil.rmtree(path, ignore_errors=True)

    def healthcheck(self) -> Tuple[bool, str]:
        """Verify weights + repo present and the `wasb` python imports torch (and sees a GPU on cuda)."""
        if not self.cfg.weights_path:
            return False, (
                "WASB weights path is empty ? set WASB_WEIGHTS (or indexing.wasb.weights_path)."
            )
        weights = os.path.expanduser(self.cfg.weights_path)
        if not os.path.exists(weights):
            return False, f"WASB weights not found at {weights}"
```

```
        repo_src = self._repo_src()
        if not os.path.isdir(repo_src):
            return False, (
                f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
            )
        # Fail fast on a typo'd device (the device-policy seam) ? a clear error here beats a cryptic
        # argparse rc=2 from wasb_infer.py at run time.
        if self.cfg.device not in VALID_DEVICES:
            return False, (
                f"unknown device {self.cfg.device!r} ? valid: {', '.join(VALID_DEVICES)}."
            )
        try:
            res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
        except FileNotFoundError:
            return (
                False,
                f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
            )
        except subprocess.TimeoutExpired:
            return False, "native torch healthcheck timed out."
        out = (res.stdout or "").strip()
        if res.returncode != 0:
            return (
                False,
                f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or out).strip()}",
            )
        # M4: resolve the device. Cache the probe so run_predict reuses it (no second subprocess).
        self._cuda_available = "cuda True" in out
        ctx = DeviceContext(self.cfg.device, self._cuda_available)
        if ctx.cuda_required_but_missing:
            # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never a silent
            # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
            return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
        if ctx.fell_back:
            logger.warning(
                "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
                "inference will be slow. Set indexing.wasb.device=cuda to require a GPU.",
                out,
            )
        return True, out

    def run_predict(
        self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
    ) -> Optional[str]:
        """Run WASB natively on a video. Returns the path to the trajectory CSV, or None.

        Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the WSL
        runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte the
        same downstream contract, so the parity comparison is apples-to-apples.
        """
        output_dir = os.path.abspath(output_win_dir)
        os.makedirs(output_dir, exist_ok=True)
        # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
        norm = str(video_win_path).replace("\\", "/")
        stem, _ext = os.path.splitext(os.path.basename(norm))
```

```
        # Content-derived key: same filename + different bytes -> different key (no cache reuse).
        key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
        out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

        if not self._copy_infer_script():
            logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
            return None

        weights = os.path.expanduser(self.cfg.weights_path)
        argv = [
            self.cfg.python_bin,
            "wasb_infer.py",
            "--video",
            os.path.abspath(str(video_win_path)),
        ]
        frames_dir: Optional[str] = None
        if self.cfg.stream_video:
            # Fast path: decode in-process, no PNG extraction. Output is bit-identical to disk.
            argv.append("--stream-video")
        else:
            scratch = self.cfg.scratch_dir or output_dir
            frames_dir = os.path.join(scratch, f"{key}_frames")
            argv += ["--frames_out_dir", frames_dir]
        # M4: resolve device="auto" to the effective device (cuda-if-available-else-cpu); an
        # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to wasb_infer.py.
        device = self._effective_device()
        argv += [
            "--weights",
            weights,
            "--sport",
            self.cfg.sport,
            "--device",
            device,
            "--out",
            out_csv,
        ]
        # GPU-feed tuning: pass only explicit non-zero values so the default argv (and its
        # byte-parity guarantee) is unchanged when the knobs are unset.
        if self.cfg.batch_size:
            argv += ["--batch-size", str(self.cfg.batch_size)]
        if self.cfg.prefetch_batches:
            argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]

        logger.info(
            "Running native WASB inference (device=%s%s) on %s ...",
            device,
            " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
            os.path.basename(norm),
        )
        try:
            res = self._run(argv, cwd=self._repo_src())
        except FileNotFoundError:
            logger.error(
                "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
            )
            return None
        except subprocess.TimeoutExpired:
            logger.error(
                "WASB native inference timed out after %ss.", self.cfg.timeout_sec
            )
```

```
                return None
            if res.returncode != 0:
                logger.error(
                    "WASB native inference failed:\n%s",
                    (res.stderr or res.stdout or "")[-2000:],
                )
                return None
            if not os.path.exists(out_csv):
                logger.error(
                    "WASB native finished but expected CSV not found at %s", out_csv
                )
                return None
            logger.info("WASB trajectory CSV: %s", out_csv)

            # Success ? the CSV is the durable output; the decoded frame cache is a pure intermediate.
            # On any earlier failure we return above WITHOUT cleaning, so the cache survives a resume.
            # (Streaming never materializes frames, so there is nothing to drop on that path.)
            if not self.cfg.keep_frames and frames_dir:
                logger.info(
                    "Cache hygiene: removing frame cache for %s "
                    "(set indexing.wasb.keep_frames=true to retain).",
                    key,
                )
                self._rm_rf(frames_dir)
            return out_csv

        # Trajectory parsing + windowing are inherited from tracknet_runner (model-agnostic).
        parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
        trajectory_to_action_windows = staticmethod(
            TrackNetRunner.trajectory_to_action_windows
        )

```

5. assistant (2026-07-03T19:48:29.889Z)

I have the full picture from both files. Let me look at the config models and presets to confirm the knob surface and how the runner is invoked per-job, plus check whether there's any existing warmup/persistence machinery.

6. user (2026-07-03T19:48:32.807Z)

```
"""Pydantic models for the application configuration.

This module defines the canonical, validated, typed configuration for the
Badminton Highlight Indexer. Built-in defaults live here (single source of truth).

Unknown / typo'd config keys are rejected at load time via extra="forbid".
"""

from __future__ import annotations

from typing import Any, List, Literal, Optional

from pydantic import BaseModel, Field, field_validator, model_validator

Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]


class ValidationRelevanceConfig(BaseModel):
    court_green_lower: list[int] = Field(default=[35, 40, 40])
    court_green_upper: list[int] = Field(default=[85, 255, 255])
```

```
    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


class ValidationConfig(BaseModel):
    min_resolution_width: int = 1280
    min_resolution_height: int = 720
    min_fps: float = 29.9
    min_blur_threshold: float = 20.0
    max_scene_cuts_per_minute: float = 0.5
    pts_max_gap_seconds: float = Field(
        default=10.0,
        ge=0.0,
        description=(
            "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
            "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
            "a gap >10s suggests edited/missing content. Mobile phones, screen "
            "recordings, and other encoders may use different GOP sizes ? tune "
            "this threshold for your recording device."
        ),
    )
    relevance: ValidationRelevanceConfig = Field(
        default_factory=ValidationRelevanceConfig
    )


class TrackNetConfig(BaseModel):
    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/TrackNetV4"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "TrackNetV4"
    weights_path: str = ""
    queue_length: int = 5
    stage_in_wsl: bool = True
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
    # decoded frame cache) are deleted automatically ? they are pure intermediates,
    # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
    # true only for debugging. On FAILURE they are always kept so a re-run can resume.
    keep_frames: bool = False
    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
    # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS, not warns).
    # 0 = no extra floor (the headroom guard still applies).
    min_free_gb: float = 0.0


class WasbIndexConfig(BaseModel):
    """Typed config for the live WASB shuttle substrate (indexing.wasb).

    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
    these knobs actually take effect ? previously this whole block was silently dropped
    because nested config sections defaulted to extra='ignore'.
    """

    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/WASB-SBDT"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "wasb"
    weights_path: str = (
        "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
    )
    sport: str = "badminton"
```

```
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
    # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
    # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU fallback
    # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU downgrade).
    device: str = "cuda"  # auto | cuda | mps | cpu
    python_bin: str = (
        "python"  # the `wasb` conda env python (invoked by path, no `conda activate`)
    )
    # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
    # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
    # from the source. Set true only for debugging. On FAILURE they are kept so a re-run resumes.
    keep_frames: bool = False
    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
    # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS, not warns).
    # 0 = no extra floor (the headroom guard still applies).
    min_free_gb: float = 0.0
    # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
    # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs for a
    # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical to the
    # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream == WSL disk,
    # 1194/1195 frames byte-identical; native run-to-run byte-identical / deterministic).
    # Tri-state:
    #   None       = per-runner default ? the WSL runner keeps DISK extraction (unchanged Windows
    #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING (the perf win).
    #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2 can't seek).
    stream_video: Optional[bool] = None
    # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
    # 2026-07-03: the default batch 8 with single-process loading left the GPU <1% utilized ?
    # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8, no
    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts into
    # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
    batch_size: Optional[int] = None
    # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
    # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
    prefetch_batches: Optional[int] = None


class FusionConfig(BaseModel):
    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

    Every default reproduces control behaviour: the fusion segmenter persists the
    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
    true ? the person-cue features, but it does NOT gate the served handoff unless a
```

```
    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
    supplied ? off-court persons (spectators / adjacent courts) are excluded.
    """

    # Which trajectory substrate seeds the windows (shuttle stays primary).
    substrate: Literal["wasb", "tracknet"] = "wasb"
    # Windowing velocity threshold for the fusion family (the engine reads
    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
    velocity_threshold: float = 0.02
    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
    # enabled ? so the default path never imports torch / touches the GPU.
    cue_flags: dict[str, bool] = Field(default_factory=dict)
    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-scoring).
    min_crossings: int = Field(default=0, ge=0)
    # Served Gate B: when the person cue is on, drop windows with median in-court players
    # < this. 0 = OFF.
    min_players: int = Field(default=0, ge=0)
    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
    court_polygon: Optional[list[list[float]]] = None
    # Net line for the person two-sided-motion split (person features only ? the shuttle
    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
    net_axis: Literal["x", "y"] = "y"
    net_position: float = Field(default=0.5, ge=0.0, le=1.0)


class IndexingConfig(BaseModel):
    default_segmenter: str = "yolo_hybrid"
    use_gpu: bool = True
    # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
    # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
    # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
    # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
    # RALLY_DETECTOR_IMPL env var overrides this. See pipeline/detectors/{stub,native_wasb}_runner.py.
    detector_impl: str = "auto"
    motion_threshold: float = 0.08
    min_segment_duration: float = 3.0
    dead_time_merge_gap: float = 2.0
    chunk_duration: float = 90.0
    chunk_overlap: float = 10.0
    detector_model: str = "fasterrcnn_resnet50_fpn"
    detector_score_threshold: float = 0.5
    yolo_velocity_threshold: float = 0.15
    yolo_frame_skip: int = 3
    yolo_tracker: str = "bytetrack.yaml"
    yolo_prefetch_workers: int = 2
    min_action_window_duration: float = 2.0
    # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0 = a window
    # longer than this many seconds is recursively split at its largest internal inactivity gap (?
    # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't swallow
    # several rallies + the dead time between them. Never merges, only cuts (recall can't drop).
    # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip. Measured n=13
```

```
        # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0, p=0.0005; cap=6
        # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate 0.651?0.161). ***
        max_window_duration: float = 6.0
        window_min_split_gap: float = (
            0.5  # min internal gap (s) that qualifies as a split point
        )
        # HARD-CAP fallback for W1: when True, an over-long window with NO internal inactivity gap
        # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob) is
        # chopped at its midpoint until ? max_window_duration, making the cap a true upper bound.
        # Only cuts (recall can't drop). OFF by default until measured/promoted.
        window_hard_cap: bool = False
        # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving slowly
        # (picked up / walked) above velocity_thresh, so the window over-extends its END (the measured
        # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose trailing
        # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only cuts.
        # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but significant
        # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13). The W1 cap
        # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md "R1". ***
        window_inactivity_end: float = 1.5
        inactivity_rally_thresh: float = 0.20
        inactivity_min_density: float = 0.30
        # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction denominator is
        # the COUNT of active samples in the trailing window, which sparse tracking inflates (one fast
        # sample after a gap ? 100% dense ? tail held open). When True, the denominator is the window's
        # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-span), so it only
        # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-set A/B at the
        # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010, sign 10W/4L, NO
        # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate ?0.010 (p=0.016),
        # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ? promote_if_served_safe(structural)=True.
        # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
        inactivity_temporal_density: bool = True
        # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active blob: after
        # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group STILL longer
        # than window_blob_factor × max_window_duration (no internal gap, no rally-end signal ? e.g.
        # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as a FORCED cut
        # (logged + flagged on the window) so the degenerate single-window outcome is avoided and the
        # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from window_hard_cap
        # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER R4 and only
```

```
    # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
    window_blob_fallback: bool = False
    # Magnitude gate that keeps R5 surgical: only a group longer than this × max_window_duration is a
    # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual is ~11×).
    # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-active
    # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md "R5".
    window_blob_factor: float = 4.0
    log_yolo_candidates: bool = True
    store_yolo_candidates: bool = True
    ai_handoff_padding: float = 2.0
    ai_max_workers: int = 5
    # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
    # chunks of a high-bitrate (4K) source blow past the provider upload cap
    # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
    # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
    # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
    # Matches gemini_refine's --clip-scale-width default.
    ai_chunk_scale_width: int = 1280
    # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
    # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff entirely and
    # emit their local candidate windows AS the rallies ? no provider, no API key, nothing
    # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the Gemini
    # path, or to operate with zero cloud dependency). The pure `gemini` segmenter ignores this.
    skip_ai_handoff: bool = False
    # Optional debug knob: trim every video to the first N seconds before processing.
    debug_duration: Optional[float] = None

    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
    fusion: FusionConfig = Field(default_factory=FusionConfig)

    @field_validator("default_segmenter")
    @classmethod
    def segmenter_must_be_registered(cls, v: str) -> str:
        # Lazy import: the segmenter registry imports modules that may import
        # config, so we resolve it at validation time rather than at module load.
        from backend.pipeline.segmenters.base import segmenter_registry

        if segmenter_registry.get(v) is None:
            available = sorted(segmenter_registry.list_available().keys())
            raise ValueError(
                f"Segmenter '{v}' is not registered. Available: {', '.join(available)}"
            )
        return v

    @model_validator(mode="after")
    def _chunk_step_must_be_positive(self) -> "IndexingConfig":
        # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-positive
        # step makes `while t < duration: t += step` loop forever, growing memory before any
        # GPU work. Reject the bad config at load time instead.
        if self.chunk_overlap >= self.chunk_duration:
            raise ValueError(
                f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration "
                f"({self.chunk_duration}); otherwise chunking never advances."
```

```
            )
        return self


class OutputConfig(BaseModel):
    default_highlights_name: str = "highlights.mp4"
    db_name: str = "sports_indexer.db"
    # Directory where the DB, highlight reels, and processed clips are written.
    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
    output_dir: str = "output"


class WatermarkConfig(BaseModel):
    """Branding overlay burned into each highlight clip during the per-clip re-encode
    (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
    (under `stitching.watermark` in config.json) to turn it off. The text is fully
    configurable. The concat stage stays stream-copy (-c copy)."""

    enabled: bool = True
    text: str = "Generated by Khelsutra.guru"
    logo_path: str = ""  # optional transparent PNG overlay; "" = no logo
    font_path: str = ""  # optional .ttf; "" = use the fontconfig `font` family below
    font: str = "Sans"  # fontconfig family used when font_path is empty
    font_size_ratio: float = Field(
        default=0.035, gt=0.0, le=0.5
    )  # text height as a fraction of frame height
    opacity: float = Field(default=0.85, ge=0.0, le=1.0)
    box: bool = True  # faint backing box behind the text for legibility
    margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
    position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
        "bottom-right"
    )


class StitchingConfig(BaseModel):
    begin_padding: float = 0.5
    end_padding: float = 0.5
    use_cache: bool = False
    min_shot_count: int = 1
    # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose duration
    # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
    # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on the
    # fully-local no-AI path, whereas duration is derived from the rally's own start/end. The
    # local detector over-fires and its false positives are mostly SHORT (motion blips,
    # background-court fragments), so this keeps the reel to the eye-catching long rallies.
    # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
    min_rally_duration: float = Field(default=0.0, ge=0.0)
    watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


class LoggingConfig(BaseModel):
    """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""

    # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
    # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
    # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
    # them to WARNING; set true to restore full chatter for request-flow debugging.
    verbose_http: bool = False
```

```
class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]

    # --- M9 edge auth (Cloudflare Access, D9) ---
    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-Assertion`
    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the identity
    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF Access
    # application's audience tag. NO secrets here ? these are public identifiers.
    edge_auth_enabled: bool = False
    cf_team_domain: str = (
        ""  # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
    )
    cf_access_aud: str = ""  # the CF Access application AUD tag the token must carry
    # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
    # backend/main.py so importing the app does no filesystem I/O; default ["*"]. Credentials stay
    # OFF ? Cf-Access is a header, not a cookie.)


class StorageConfig(BaseModel):
    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
    behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO Spaces
    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a locally MOUNTED
    Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the constructor ?
    **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
    (boto3/google auth chains supply credentials). See ``backend/storage/``."""

    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
    # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
    output_root: str = ""
    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output `backend`/`output_root`).
    # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read IN PLACE
    # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a bare/relative
    # input name is resolved against ``input_root`` (e.g. ``input_root="gs://bucket/videos"`` + "x.mp4"
    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
```

```
``input_backend``
    # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the input
    # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are fetched to a
    # scratch dir before processing (see ``backend.storage.acquire_local_input``).
    input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
    input_root: str = ""
    # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
    # One folder per video keyed by the MD5 video_id: <root>/[<workspace_prefix>/]<video_id>/...
    # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
    # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5). When OFF
    # the legacy flat layout is used unchanged.
    single_folder_per_video: bool = False
    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
    workspace_root: str = ""
    # Cloud namespace prefix under the root so per-video folders sit tidily beside
    # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root directly.
    workspace_prefix: str = "workspaces"
    local: dict[str, Any] = Field(default_factory=dict)
    s3: dict[str, Any] = Field(default_factory=dict)
    gcs: dict[str, Any] = Field(default_factory=dict)
    gdrive: dict[str, Any] = Field(default_factory=dict)
    # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the hyphen?underscore lookup in
    # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive scope.
    gdrive_api: dict[str, Any] = Field(default_factory=dict)


class DeploymentConfig(BaseModel):
    """Deployment profile ? one switch that selects a coherent bundle of compute + storage
    knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).

    **Pure / additive:** the empty default profile (``""``) applies NO preset, so the pipeline
    behaves exactly as before this section existed. A profile is opt-in (``config.json``
    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
    *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core reads
    these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING knobs
    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.

    Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""

    # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
    profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
    # Where the core runs. Set by the profile preset; overridable in config.json. Inert in M1
    # (recorded for observability; consumed by the ComputeBackend seam from M2+).
    compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""


class BillingConfig(BaseModel):
    """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).

    **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
```

```
NULL =
    today's behaviour). When enabled, a job's cost is computed + stored in **USD** via the versioned
    ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default pricebook
    version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the owner
    inserts a real, calibrated version and points ``pricebook_version`` at it.

    ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a follow-up
    wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the ledger +
    pricebook + the recording seam; usage emission (the metering hooks) is a separate step."""

    enabled: bool = False
    pricebook_version: str = (
        "placeholder-v0"  # the seeded $0 version; owner flips to a real one
    )
    fx_inr_per_usd: float = 83.0  # USD?INR display rate (Bangalore market)
    margin: float = 0.0  # resale markup (?0); 0 = bill at cost


class FlywheelConfig(BaseModel):
    """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the consent?capture?eval?goldenize
    loop: on a CONSENTED adult job, durably **capture** the source + rally output into the per-video
    workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden TRAINING set, so
    the owned model climbs toward the D11 ship floor.

    **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-identical to
    today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even ``enabled=True``
    captures **nothing**, because ``flywheel.consent_attested`` currently returns ``False`` for every
    job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-delete-raw).
    This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent columns ?
    wiring) + the owned-model + golden-label data needed to *activate* shadow-eval/promotion are
    owner/counsel/GPU scope. Pairs with M9-consent."""

    enabled: bool = False
    workspace_prefix: str = (
        "flywheel"  # per-video workspace prefix for captured artifacts
    )
    shadow_eval: bool = True  # run the owned-vs-Gemini shadow comparison on capture
    min_promote_n: int = 5  # promote_if_served_safe `min_n` (the served-safe gate)


class AppConfig(BaseModel):
    """Root configuration object.

    All runtime configuration should be accessed via an instance of this class
    (or the loader) rather than raw dict lookups.
    """

    sport: Sport = "badminton"
    ai_provider: str = "gemini"
    # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
```

```
flagship;
    # the alias never deprecates underneath us. Eval reports record the run date alongside.
    ai_model: str = "gemini-flash-latest"

    validation: ValidationConfig = Field(default_factory=ValidationConfig)
    indexing: IndexingConfig = Field(default_factory=IndexingConfig)
    output: OutputConfig = Field(default_factory=OutputConfig)
    stitching: StitchingConfig = Field(default_factory=StitchingConfig)
    security: SecurityConfig = Field(default_factory=SecurityConfig)
    logging: LoggingConfig = Field(default_factory=LoggingConfig)
    storage: StorageConfig = Field(default_factory=StorageConfig)
    deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
    billing: BillingConfig = Field(default_factory=BillingConfig)
    flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)

    # Reject unknown keys ? typed config migration is complete (D3).
    model_config = {
        "extra": "forbid",
        "validate_assignment": True,
    }


# Convenience type alias for callers that want to annotate
Config = AppConfig

```

7. user (2026-07-03T19:48:33.971Z)

```
"""Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING M1).

A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).

Precedence (lowest ? highest), realised by the loader::

    built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set

This module is **pure data + pure functions** ? no IO, and (critically) **no torch import at
module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
(``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15: cloud
spend is never entered silently).
"""

from __future__ import annotations

import copy
import os
from typing import Any, Mapping, Optional

# Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal`` mirrors
# these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")

# Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
# when config.json explicitly overrides compute_target into a state inconsistent with
```

```
the profile.
COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
    "truly-local": "local_gpu",
    "cloud-serving": "cloud_run",
    "cpu-mock": "cpu_mock",
}

# A preset is a partial, nested override dict over the model defaults. It touches ONLY knobs
# that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}`` plus
# ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones extend
# these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
# Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
PROFILE_PRESETS: dict[str, dict[str, Any]] = {
    "truly-local": {
        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
        # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
        # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
        "indexing": {"skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
    "cloud-serving": {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
        # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid L4
        # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
        # CPU-feed-bound. Serving opts into the measured fast values; local/WSL unchanged.
        "indexing": {
            "detector_impl": "native",
            "skip_ai_handoff": False,
            "wasb": {"batch_size": 64, "prefetch_batches": 4},
        },
        "storage": {"backend": "gcs"},
    },
    "cpu-mock": {
        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
        "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
}


def is_known_profile(profile: str) -> bool:
    """True for a non-empty, recognised profile name."""
    return bool(profile) and profile in PROFILE_PRESETS


def preset_for(profile: str) -> dict[str, Any]:
    """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
    module-level table), or ``{}`` for ``""`` / an unknown profile."""
    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


def _truthy(val: Optional[str]) -> bool:
    """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
    if val is None:
```

```
            return False
        return val.strip().lower() in {"1", "true", "yes", "on"}


    def _cuda_available() -> bool:
        """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here and
        ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no GPU'."""
        try:
            # Intentionally lazy + local: keeps the config-load path torch-free (torch is heavy
            # and optional). Only reached when a caller opts into probe_cuda.
            import torch

            return bool(torch.cuda.is_available())
        except Exception:
            return False


    def probe_cuda_available() -> bool:
        """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).

        Exposed for the per-profile startup checks (``backend/config/startup.py``) so a caller
        that *wants* the GPU heads-up can pass the result in without the startup module importing
        torch itself. Best-effort by design: if torch lives only in a detector subprocess env
        (native WASB), this returns ``False`` in the main process ? which is why the startup checks
        treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard
        error that could false-fail."""
        return _cuda_available()


    def suggest_profile(
        env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
    ) -> str:
        """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.

        Signals, in priority order:

          - ``K_SERVICE`` set (Cloud Run)         ? ``"cloud-serving"``
          - ``CI`` truthy                         ? ``"cpu-mock"``
          - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
          - otherwise                             ? ``""`` (no strong suggestion)

        ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and never
        imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe passes
        ``probe_cuda=True``. The result is advisory only: the caller decides whether to act on it.
        """
        e = os.environ if env is None else env
        if e.get("K_SERVICE"):
            return "cloud-serving"
        if _truthy(e.get("CI")):
            return "cpu-mock"
        if probe_cuda and _cuda_available():
            return "truly-local"
        return ""
```

8. user (2026-07-03T19:48:34.679Z)

```
# Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7, ADR-012/D16).
#
# Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-located so
# CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the owner on the
# M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too heavy) ? the
# dispatch shim is proven with a fake client (tests/test_compute_backend.py).
#
# Two python stacks, deliberately separate (a framework conflict otherwise ? same split as the
# native GPU box, deploy/digitalocean/gpu_setup.sh):
#   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
#   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as a subprocess
#     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-installs with it.
#
# ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT cleared for
#   commercial sharing. They are NOT baked into this image; stage them at runtime (a bucket fetch or a
#   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner serving.

FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04

ENV DEBIAN_FRONTEND=noninteractive \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    CONDA_DIR=/opt/conda \
    WASB_REPO_DIR=/opt/models/WASB-SBDT

# ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB env build.
RUN apt-get update && apt-get install -y --no-install-recommends \
        ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
        python3 python3-pip python3-venv \
    && rm -rf /var/lib/apt/lists/*

# --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a, ADR-005) ---
# Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ? stays
# MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's Anaconda-ToS-gated
# `defaults` channels (the ?200-employee Terms-of-Service trap flagged in PACKAGING_PLAN.md §3).
# MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so $WASB_PYTHON below
# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are unaffected by the switch.
ENV MAMBA_ROOT_PREFIX=/opt/conda
RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
        | tar -xj -C /usr/local bin/micromamba \
    && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
    && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url https://download.pytorch.org/whl/cu113 \
    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
```

```
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
        && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
            hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib "numpy<1.24"

    # --- Main application env ---
    WORKDIR /app
    COPY pyproject.toml README.md ./
    COPY requirements-trackers.txt ./
    COPY backend ./backend
    COPY training ./training
    # Install with the [storage-gcs] extra: the cloud worker PULLS the gs:// input and PUSHES the
    # gs:// outputs/done-marker through backend.storage (google-cloud-storage is lazy-imported there).
    # Without it the job dies at the first gs:// fetch with `ModuleNotFoundError: google.cloud` ? the
    # real-Cloud-Run failure the mock-client tests can't see (the bucket I/O is faked in tests).
    # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
    # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band before
    # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable git SHA is
    # single-sourced in requirements-trackers.txt.
    RUN python3 -m pip install --upgrade pip \
        && python3 -m pip install -r requirements-trackers.txt \
        && python3 -m pip install ".[storage-gcs]"

    # The native runner finds the WASB env + repo by env (no `conda activate` needed); weights come in at
    # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the default.
    # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until process
    # exit, so Cloud Logging shows a single end-of-run burst instead of live progress (observed on the
    # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log nothing.
    ENV PYTHONUNBUFFERED=1 \
        WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
        WASB_REPO_DIR=/opt/models/WASB-SBDT \
        WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
        RALLY_DETECTOR_IMPL=native

    # The dispatcher (CloudRunCompute) supplies the per-execution args:
    #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid --set ?
    ENTRYPOINT ["python3", "-m", "backend.worker"]

```

9. assistant (2026-07-03T19:48:39.508Z)

I now have the complete picture. Let me confirm two specifics: (1) whether `model.eval()` / grad state is set inside `build_detector` / `run_tensor` (which I cannot read, but can reason about from the WASB call surface and the `torch.no_grad()` scoping in wasb_infer.py), and (2) whether the worker/dispatch uses a Cloud Run Job (ephemeral, one job per video) vs a Service. Let me grep the worker and compute backend for the job lifecycle.

10. user (2026-07-03T19:48:43.856Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\models.py:243:    # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the Gemini
```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\ui\assets\index-Cqr7zJeJ.js:60:[Omitted long matching line]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\person_detector.py:102:   self.model.eval()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\person_detector.py:152:   with torch.no_grad():
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\wasb_infer.py:619:
stack.enter_context(torch.no_grad())

11. user (2026-07-03T19:48:45.612Z)

Found 32 files
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\presets.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\native_wasb_runner.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\models.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\worker__main__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\startup.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\utils\run_telemetry.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\utils\app_home.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\ui\assets\index-Cqr7zJeJ.js
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\reporting\spec.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\results.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\providers\gemini.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\segmenters\base.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\segmenters\tracknet_hybrid.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\segmenters\trajectory_hybrid.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\segmenters\wasb_hybrid.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\stub_runner.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\tracknet_runner.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\wasb_runner.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\main.py

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\orchestration.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\base.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\pipeline\detectors\device.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\infrastructure\execution_policy.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\infrastructure\database_jobs.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\infrastructure\database.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\loader.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\compute\cloud_run.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\compute__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\compute\base.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\api\static\app.js
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\api\job_worker.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\annotations\base.py

12. assistant (2026-07-03T19:48:49.467Z)

The grep confirms the WASB detector's `build_detector`/`run_tensor` are opaque (in the WASB repo, not readable), and the only grad/eval control in wasb_infer.py is the `torch.no_grad()` at line 619. Let me confirm the Cloud Run Job (ephemeral, one-execution-per-video) lifecycle in the compute backend and worker entry.

13. user (2026-07-03T19:48:51.038Z)

```
"""``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and bucket-poll
for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).

Flow (the cloud-serving compute path):
  1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m backend.worker infer
     --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4 GPU).
  2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16 ? the
     completion signal is a tiny marker object the worker writes, NOT the Cloud Run API status, so
     it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero (D7): the
     cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can undercut the
     job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-healthy
     worker that finished just past the deadline is rescued, not wasted ? then best-effort CANCELS
     the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
```

```
  3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` + native detector)
so it never recursively re-dispatches ? the dispatcher decides cloud, the container just computes.

The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole shim is unit-tested
with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run`` and is
owner-verified on the M7 real run.
"""

from __future__ import annotations

import json
import logging
import os
import tempfile
import uuid
from abc import ABC, abstractmethod
from typing import Any, List, Optional

from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
from backend.infrastructure.execution_policy import (
    DEFAULT_POLICY,
    ExecutionPolicy,
    PolicyTimeout,
    poll_until,
)
from backend.storage import fetch, get_storage, parse_uri

LOG = logging.getLogger("compute.cloud_run")

# The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch branch.
# (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB runner.)
_DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
    "deployment.compute_target=local_gpu",
    "indexing.detector_impl=native",
)


class CloudRunJobClient(ABC):
    """Triggers a Cloud Run **Job** execution with per-execution container arg overrides.

    Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real implementation
    overrides the job's container ``args`` for this execution and starts it."""

    @abstractmethod
    def run_job(
        self,
        job_name: str,
        argv: List[str],
        *,
        region: str,
        project: Optional[str] = None,
    ) -> str:
        """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return an
```

```
            execution id/name (for logs). Does NOT block on completion (the caller bucket-polls)."""

        @abstractmethod
        def cancel_execution(self, execution: str) -> None:
            """Cancel a running execution by the name ``run_job`` returned. Called when the bucket-poll
            times out so the abandoned execution doesn't keep billing the GPU until ``--task-timeout``.
            May raise ? the caller treats every failure as best-effort (the original poll timeout is
            NEVER masked by a cancel error)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test dependency)."""

        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            # A None/empty project would build a bogus "projects/None/locations/..." path (job_path
            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is exercisable
            # in CI (where google-cloud-run is absent) and so mypy narrows project to str for job_path.
            if not project:
                raise RuntimeError(
                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP project id "
                    "to resolve the job path; set it on the control plane (see get_compute_backend)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.JobsClient()
            )  # pragma: no cover - exercised only on the real M7 run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
```

```
        )

    def cancel_execution(self, execution: str) -> None:
        # run_job falls back to the literal string "execution" when the operation metadata carries
        # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK import
        # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI (where
        # google-cloud-run is absent).
        if "/executions/" not in (execution or ""):
            raise RuntimeError(
                f"not a fully-qualified Cloud Run execution name: {execution!r} ? expected "
                "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not resolve "
                "one from the operation metadata, so there is nothing to cancel by name)."
            )
        try:
            from google.cloud import run_v2  # type: ignore[attr-defined]
        except Exception as exc:  # pragma: no cover - real SDK not present in CI
            raise RuntimeError(
                "google-cloud-run is required to cancel a Cloud Run execution; install it on the "
                "control plane (it is intentionally NOT a CI/test dependency)."
            ) from exc
        client = (
            run_v2.ExecutionsClient()
        )  # pragma: no cover - exercised only on a real run
        client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))


class CloudRunCompute(ComputeBackend):
    """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

    def __init__(
        self,
        *,
        run_client: CloudRunJobClient,
        job_name: str,
        region: str,
        project: Optional[str] = None,
        storage_config: Optional[dict] = None,
        policy: ExecutionPolicy = DEFAULT_POLICY,
        token: Optional[str] = None,
        container_overrides: Optional[tuple[str, ...]] = None,
    ) -> None:
        self._client = run_client
        self._job_name = job_name
        self._region = region
        self._project = project
        self._storage_config = storage_config or {}
        self._policy = policy
        # The dispatch token keys the done-marker so concurrent jobs to one bucket don't collide.
        # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic (tests).
        self._token = token
        self._container_overrides = (
            _DEFAULT_CONTAINER_OVERRIDES
            if container_overrides is None
            else tuple(container_overrides)
        )

```

```
    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
        out_root = spec.out_uri.rstrip("/")
        token = self._token or uuid.uuid4().hex
        done_uri = f"{out_root}/_dispatch/{token}.done"
        argv: List[str] = [
            "infer",
            "--video-uri",
            spec.video_uri,
            "--out-uri",
            spec.out_uri,
            "--done-uri",
            done_uri,
        ]
        if spec.segmenter:
            argv += ["--segmenter", spec.segmenter]
        for kv in (*spec.config_overrides, *self._container_overrides):
            argv += ["--set", kv]

        execution = self._client.run_job(
            self._job_name, argv, region=self._region, project=self._project
        )
        LOG.info(
            "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
            self._job_name,
            execution,
            done_uri,
        )

        try:
            marker = poll_until(
                lambda: self._read_marker(done_uri),
                self._policy,
                label="cloud-run-infer",
                logger=LOG,
            )
        except PolicyTimeout as timeout_exc:
            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
        video_id = str(marker.get("video_id", ""))
        # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-unreadable /
        # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
        rc = int(marker.get("returncode", 1))
        if not video_id:
            LOG.warning(
                "cloud-run: job=%s completion marker present but empty/unreadable (no "
                "video_id) ? treating as failure",
                self._job_name,
            )
            rc = rc or 1
        outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
        report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
        LOG.info(
            "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
        )
        return ComputeResult(
            returncode=rc,
            outputs_uri=outputs_uri,
            video_id=video_id,
            report=report,
            execution=str(execution or ""),
        )

    def _handle_poll_timeout(
        self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
```

```
    ) -> dict:
        """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

        The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed job's own
        ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past the deadline:
        ONE final marker check rescues that completed GPU run instead of wasting it. If the marker
        is genuinely absent, the execution is still running ? best-effort cancel it (otherwise the
        L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout`` carrying the
        execution name so the operator can verify the state in the GCP console. A cancel failure
        NEVER masks the original timeout."""
        try:
            late = self._read_marker(done_uri)
        except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence ? failure
            LOG.warning(
                "cloud-run: final post-timeout marker check failed ? treating as absent",
                exc_info=True,
            )
            late = None
        if late is not None:
            LOG.warning(
                "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check "
                "? rescuing the completed result",
                self._job_name,
                execution,
            )
            return late
        try:
            self._client.cancel_execution(execution)
        except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the timeout
            cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
            LOG.warning(
                "cloud-run: best-effort cancel of execution=%s failed: %s",
                execution,
                cancel_exc,
                exc_info=True,
            )
        else:
            cancel_note = "cancel requested"
            LOG.info("cloud-run: requested cancel of execution=%s", execution)
        raise PolicyTimeout(
            f"{timeout_exc} [cloud-run job={self._job_name} "
            f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution "
            "state in the GCP console]"
        ) from timeout_exc

    # --- storage-backed completion signal (bucket-poll) ---
    def _read_marker(self, done_uri: str) -> Optional[dict]:
        """The done-marker dict if the worker has written it, else None ? poll_until keeps waiting."""
        ref = parse_uri(done_uri)
        if not get_storage(ref.backend, config=self._storage_config).exists(ref):
            return None
        return (
            self._read_json(done_uri) or {}
```

```
        )  # empty dict still terminates the poll (present == done)

    def _read_json(self, uri: str) -> dict:
        # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst ? a None dst
        # raises ValueError ? unlike local's identity passthrough. Without an explicit dst this read
        # would silently return {} for every gs://|s3:// object (the only backends this shim targets).
        try:
            ref = parse_uri(uri)
            with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
                dst = os.path.join(td, ref.name or "obj.json")
                local = fetch(uri, dst, config=self._storage_config)
                with open(local, "r", encoding="utf-8") as f:
                    data: Any = json.load(f)
            return data if isinstance(data, dict) else {}
        except Exception:
            LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
            return {}

```

14. user (2026-07-03T19:48:51.930Z)

```
"""Abstract base for detector runners (TrackNet, WASB, future native, etc.).

This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.

The ABC lets us:
- Keep the current WSL adapters working unchanged for Windows users.
- Swap in (or A/B test) a Linux-native implementation later with zero changes to
  the hybrid segmenters or the model-agnostic windowing brain.
- Provide a single place to evolve the healthcheck / error contract and thread
  detector readiness into per-video reports.

Contract (deliberately matches what the concrete runners + hybrids + tests already did):
- run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
  Run detector, return the Windows-side path to the produced trajectory CSV
  (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
  The caller (hybrid) is responsible for output dir; impls may stage the video
  into WSL fs for perf.

- healthcheck() -> tuple[bool, str]
  Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
  Message is logged (and can be surfaced to reports or UX in future).
  Intended to be cheap (env existence, import, GPU visibility, weights present).

Existing behavior is 100% preserved. The only additions are the inheritance and
the explicit common type for future implementers.

How to add a new runner (for docs / future contributors):
1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
2. Implement run_predict and healthcheck (return (bool, str) tuple).
3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
   TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
   deliberately static and model-agnostic). If the new detector needs different
   windowing, it can implement its own or we generalize later.
4. Lazy-import heavy native deps inside the methods (so importing the segmenter
   registry never pulls torch/tf on a machine that doesn't have them).
5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
   DetectorRunner instances (they already do the right thing via duck-typing + the
   type hints we add here).
6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
```

```
   returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
7. No change to eval/calibration code paths (they consume the CSV + the static
   windowing helpers directly).

Optional Linux-native path (the main de-risk goal):
A LinuxNativeRunner (or ONNXRunner) would:
- Skip all wsl / _stage / conda activate.
- Load weights locally (torch or onnx) or call a native binary.
- Write the exact same CSV format so trajectory_to_action_windows works verbatim.
- healthcheck can just check "weights file exists and torch importable + CUDA?".
This removes the WSL requirement for Linux users / future SaaS workers while
keeping the exact same candidate quality characteristics (or better, once native
weights are available).

Error surfacing into reports:
The healthcheck message is already logged by the hybrids on failure. The reporting
harvest sees the outcome as "0 candidates" + detector name. When we wire richer
segmenter Results (or pass detector_health into emit_indexer_report context), the
(ok, msg) can be recorded explicitly under segmentation.details or a new detector
stage. The ABC makes that future change local to one place.

See also:
- backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
- tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
- docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
"""

import logging
import os
import shlex
import subprocess
from abc import ABC, abstractmethod
from typing import Any, Callable, List, Optional, Tuple

logger = logging.getLogger(__name__)

# Frame extraction of a high-res clip can materialise ~15× the source size in PNGs before the
# detector consumes them (STUDIO_MEDIA_LIFECYCLE.md D11). The WSL stage fs must hold that peak.
WSL_FRAME_HEADROOM = 15.0


def wsl_tilde_quote(path: str) -> str:
    """shlex.quote that still lets bash expand a leading ``~/``.

    ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
    treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
    directory" ? this broke live WSL staging for the default ``~/clips``
    stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
    "shlex-quote breaks ~ expansion" item). Quoting only the part after
    ``~/`` keeps expansion AND protects spaces/specials in the remainder.
    """
    if path == "~":
        return path
    if path.startswith("~/"):
        return "~/" + shlex.quote(path[2:])
    return shlex.quote(path)


class WslCommandMixin:
    """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.

    Lives beside (not on) DetectorRunner: a future Linux-native runner must not
    inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
```

```
    and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
    ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
    """

    cfg: Any

    def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
        full = f"source {self.cfg.conda_sh} && {inner_cmd}"
        argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
        logger.debug("WSL exec: %s", full)
        return subprocess.run(
            argv, capture_output=True, text=True, timeout=(self.cfg.timeout_sec or None)
        )

    # ---- cache hygiene helpers (shared by both WSL runners) ------------------
    def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
        """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.

        Returns None if it can't be determined (never raises). Used as a pre-run
        disk guard so a long extraction doesn't fill the disk mid-flight.
        """
        try:
            res = self._wsl_bash(
                f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk 'NR==2{{print $4}}'"
            )
        except (subprocess.TimeoutExpired, OSError):
            return None
        try:
            return int((res.stdout or "").strip()) / (1024**3)
        except (ValueError, AttributeError):
            return None

    def _wsl_frame_stage_shortfall(
        self, source_win_path: str, *, headroom: float = WSL_FRAME_HEADROOM
    ) -> Optional[str]:
        """Actionable error string if the WSL stage fs lacks ~``headroom``× the source size for frame
        extraction, else ``None`` (STUDIO_MEDIA_LIFECYCLE.md D11 ? BLOCK the local/WSL frame stage).

        BLOCKING replaces the old warn-only guard: extracting a 4K clip can balloon to ~15× the
        source in PNGs and fill the disk mid-run (a wasted long run that looks like a quality fail).
        Best-effort about MEASURING ? an unreadable source size or an undeterminable WSL free-read
        returns ``None`` so we never block on inability to measure ? but strict once a real shortfall
        is seen. ``min_free_gb`` (config) still acts as an absolute floor on top of the headroom."""
        try:
            src_bytes = os.path.getsize(source_win_path)
        except OSError:
            return None
        free_gb = self._wsl_free_gb(self.cfg.wsl_stage_dir)
        if free_gb is None:
            return None
        src_gb = src_bytes / (1024**3)
        need_gb = max(src_gb * headroom, float(getattr(self.cfg, "min_free_gb", 0.0) or 0.0))
        if free_gb >= need_gb:
            return None
        return (
            f"Low WSL disk at {self.cfg.wsl_stage_dir}: {free_gb:.1f} GB free, but frame "
```

```
            f"extraction needs ~{need_gb:.1f} GB (~{headroom:g}× the {src_gb:.2f} GB source). "
            f"Free space (e.g. python -m tools.cleanup_caches) and retry."
        )

    def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
        """Best-effort recursive delete of WSL paths (post-success cleanup; never raises)."""
        paths = [p for p in wsl_paths if p]
        if not paths:
            return
        quoted = " ".join(wsl_tilde_quote(p) for p in paths)
        try:
            self._wsl_bash(f"rm -rf {quoted}")
        except (subprocess.TimeoutExpired, OSError) as e:
            logger.warning("WSL cache cleanup failed (non-fatal): %s", e)

    def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
        """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.

        Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
        filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
        fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
        trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
        A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE any
        frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
        ``cp`` still gets its chance (we never block a genuinely-readable source).
        """
        try:
            res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
        except (subprocess.TimeoutExpired, OSError):
            return None
        if "READABLE" in (res.stdout or ""):
            return None
        return (
            f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
            f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
            f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
        )


class DetectorRunner(ABC):
    """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""

    @abstractmethod
    def run_predict(
        self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
    ) -> Optional[str]:
        """Run inference and return path to trajectory CSV (or None on failure).

        ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
        videos that happen to share a filename cannot reuse each other's frames or CSV.
        It is optional for back-compat; callers in the live pipeline always pass it.
        """
        pass

    @abstractmethod
    def healthcheck(self) -> Tuple[bool, str]:
        """Quick check if the runner (env, weights, etc.) is usable.

        Returns (ok, message). Never raises. Cheap to call before a long run.
        """
        pass

```

```
        # De facto trajectory contract: the trajectory hybrids call these
        # unconditionally after run_predict. Concrete runners reuse/re-export the
        # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
        # a runner with custom windowing supplies its own with the same shapes.
        parse_trajectory_csv: Callable[..., Any]
        trajectory_to_action_windows: Callable[..., Any]

```

15. assistant (2026-07-03T19:49:03.385Z)

I now have everything I need. The architecture is fully confirmed:

Job lifecycle (the load-bearing fact): `CloudRunCompute.run_infer` (cloud_run.py:179) dispatches ONE Cloud Run **Job** execution per video via `run_job`. The Dockerfile ENTRYPOINT is `python3 -m backend.worker` and the args are `infer --video-uri ... --out-uri ...`. So each video = a fresh container = a fresh process that (a) pays lazy CUDA context init, (b) imports torch 1.11 + Hydra + builds the WASB detector via `build_detector(cfg)` which loads the `.pth.tar` weights from disk, (c) does cudnn autotune on the first forward. There is no persistent process; `build_detector` is called exactly once per `run()` invocation (wasb_infer.py:601), and `run()` is the whole process. Task #2 confirms "L4 Cloud Run **Job**" (not Service).

Grad/eval state: `torch.no_grad()` IS entered (wasb_infer.py:619, wrapping the whole detector loop). `model.eval()` is NOT visible in wasb_infer.py ? it is inside WASB's opaque `build_detector(cfg)`. WASB's own eval config path (`configs/eval`) and the fact that this is the upstream eval harness strongly implies the detector wrapper sets `.eval()` internally, but wasb_infer.py does not assert it. `set_grad_enabled` is never called explicitly; `no_grad()` covers it.

Let me do the honest cost accounting before returning the finding. The measured baseline is 727.9s for the whole Phase-2 with GPU <1% utilized. The warmup/JIT/context cost is a ONE-TIME fixed cost (a few seconds), whereas 727s is dominated by per-window feed-bound serialization across ~21k windows. So this lever is real but small relative to the headline number.

16. user (2026-07-03T19:49:42.195Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 6495 bytes): {"lever": "model-warmup-persist: (a) a dummy warmup forward at healthcheck/startup to move lazy-CUDA-context + cudnn-autotune + first-JIT off the measured path, and (b) recognizing the ephemeral Cloud
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

17. assistant (2026-07-03T19:49:44.600Z)

I sent the fields wrapped in a `raw` string. Let me call the tool with the proper structured fields directly.

18. user (2026-07-03T19:50:22.104Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 6620 bytes): {"lever": "model-warmup-persist: (a) a dummy warmup forward at healthcheck/startup so lazy CUDA-context + cuDNN autotune + first-forward JIT/kernel-cache land OUTSIDE the measured detection timer; (b)
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

19. assistant (2026-07-03T19:50:25.605Z)

Let me pass the fields as proper individual JSON fields, keeping the text concise to avoid escaping issues.

20. user (2026-07-03T19:50:55.566Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 5649 bytes): {"lever": "model-warmup-persist: (a) a warmup forward at startup so lazy CUDA-context init + cuDNN autotune + first-forward JIT/kernel-cache land OUTSIDE the measured detection timer; (b) recognizing
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

21. assistant (2026-07-03T19:51:00.085Z)

The harness is showing me a `raw` wrapper, which suggests my tool input is being double-wrapped. Let me call the tool passing each parameter as a native argument rather than a serialized JSON string, keeping content tight.

22. user (2026-07-03T19:51:31.991Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 5997 bytes): {"lever": "model-warmup-persist: (a) a warmup forward at startup so lazy CUDA-context init + cuDNN autotune + first-forward JIT/kernel-cache land OUTSIDE the measured detection timer; (b) recognizing
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

23. assistant (2026-07-03T19:51:35.375Z)

The backtick characters in my content (`` `python3 ...` ``) may be the problem, or the harness keeps interpreting my input as a `raw`-wrapped string. The tool expects native structured arguments. Let me provide them directly as proper JSON object fields with no backticks and careful escaping.

24. user (2026-07-03T19:52:13.889Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 7009 bytes): {"lever": "model-warmup-persist: (a) a warmup forward at startup so lazy CUDA-context init + cuDNN autotune + first-forward kernel-cache land OUTSIDE the measured detection timer; (b) recognizing the
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>